Final Project Report

Comparative Analysis of Modern Tabular ML for Network Intrusion Detection

Team Members: Farman H. Sayem, Jace C. Rea, Kunle S. Oguntoye

Due Date: May 13, 2026

1. Project Overview

This project evaluates modern tabular machine learning models for network intrusion detection using a reproducible Google Colab framework. The final system classifies network flows as benign or attack traffic and compares LightGBM, XGBoost, CatBoost, and a LightGBM + SMOTE variant using the CSE-CIC-IDS2018 dataset as the main benchmark. CIC-IDS2017 is then used as a cross-year drift dataset to test whether same-year model performance transfers to related but shifted network traffic.

The central problem is that intrusion detection is not just a basic accuracy task. Network traffic is imbalanced, attacks are the minority class, and deployment decisions depend on the cost of missed attacks, the cost of false alarms, and whether the model probability scores can be trusted. A model can appear strong on standard classification metrics but still be difficult to use in practice if its thresholds are poorly chosen, its probability estimates are unreliable, or its performance collapses under data drift.

The final goal was to move beyond a simple model comparison and build a reusable evaluation pipeline. The completed project covers data loading, cleaning, feature alignment, train/validation/test splitting, class imbalance handling, model training, metric evaluation, probability calibration, threshold analysis, and drift testing.

2. Motivation and Relevance

Network intrusion detection remains a practical cybersecurity challenge because malicious events are relatively rare compared with normal traffic, but the cost of missing those events can be high. A false negative may allow an intrusion to slip through, while too many false positives can overload analysts and cause alert fatigue. For this reason, accuracy alone is not enough.

This project focuses on metrics and evaluation layers that better match security use. Recall measures how many attacks are caught. Precision and false positive rates help describe analyst workload. PR-AUC is more informative than accuracy or ROC-AUC when the attack class is smaller. Brier score and reliability diagrams evaluate whether model probability outputs can support risk scoring. Drift analysis tests whether a model trained on one year of traffic remains useful when traffic distributions change.

The project is relevant because it treats intrusion detection as an operational decision problem rather than only a benchmark classification problem. The conclusion is therefore not just which model has the best score, but how model selection, calibration, threshold choice, and drift monitoring work together.

3. Work Completed and Methodology

3.1 Dataset setup and final data used

The final notebook used real CIC-style intrusion detection data with USE_REAL_DATA=True. CSE-CIC-IDS2018 served as the primary dataset, while CIC-IDS2017 served as the drift dataset. Because the full datasets are large, the notebook streamed multi-GB CSV files in chunks, stripped inconsistent column whitespace, skipped bad rows, and sampled the data to keep the run feasible in Google Colab.

Dataset	Role	Rows after cleaning	Attack rate / class split	Use in project
CSE-CIC-IDS2018	Primary modeling dataset	99,750	23.6% attack; 76,186 benign and 23,564 attack	Training, validation, same-year test evaluation
CIC-IDS2017	Drift dataset	48,902	18.7% attack after cleaning	Cross-year drift and robustness test
Final modeling feature set	Aligned feature space	23 shared numeric features	Target binarized: benign = 0; attack = 1	Same 2018-fitted preprocessing applied to 2017

Table 1. Final dataset and feature alignment summary.

The final CSE-CIC-IDS2018 sample retained 99,750 cleaned rows. The attack class was smaller than the benign class but still large enough to support supervised training. The final CIC-IDS2017 drift set retained 48,902 cleaned rows. Both datasets were reduced to a common set of 23 usable numeric features so that models trained on 2018 could be evaluated on 2017 without feature mismatch.

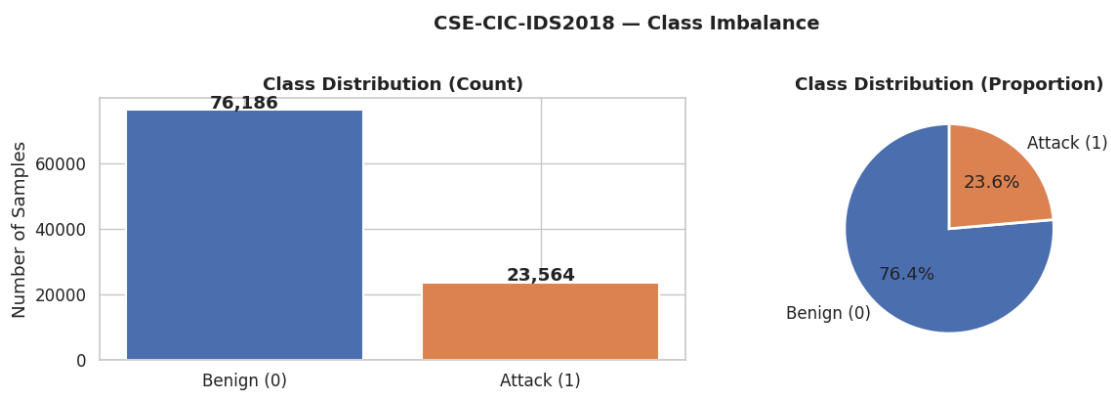

Figure 1. Final CSE-CIC-IDS2018 class distribution after cleaning.

3.2 Cleaning, preprocessing, and split design

The preprocessing pipeline was built to reduce leakage and make the experiment reproducible. Infinite values were replaced with missing values, duplicate rows were removed, identifier columns such as IP addresses, ports, timestamps, protocol, and flow ID were dropped when present, constant or unusable columns were removed, and extreme numeric outliers were clipped at the 99.9th percentile.

Missing numeric values were handled with median imputation, and features were scaled through a scikit-learn preprocessing pipeline.

A key design choice was fitting the preprocessing pipeline only on the 2018 training split. The fitted pipeline was then used to transform the validation set, test set, and CIC-IDS2017 drift set. This prevented test-set leakage and simulated the way a deployed detector would apply a fixed transformation to future traffic.

The final 2018 data was split using a stratified 70/15/15 design: 69,824 training rows, 14,963 validation rows, and 14,963 test rows. Each split preserved an attack rate of about 23.6%. The training split was used for learning and imbalance handling, the validation split was used for calibration and threshold analysis, and the test split was held out for final same-year evaluation.

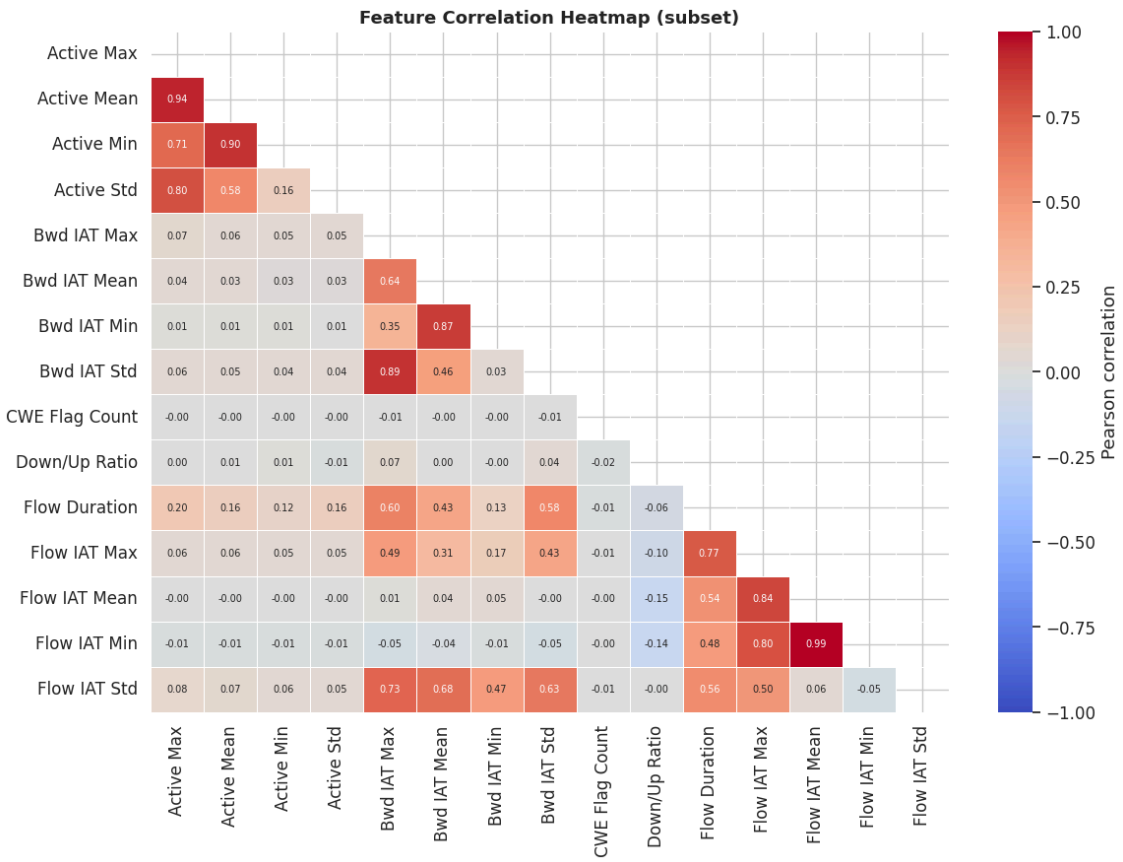

Figure 2. Correlation heatmap for selected numeric features. Several timing/IAT variables were strongly correlated.

3.3 Models, imbalance handling, and evaluation layers

The model comparison included LightGBM, XGBoost, CatBoost, and LightGBM trained after SMOTE oversampling. Class weighting was used as the primary imbalance strategy because it penalizes missed attacks without generating synthetic samples. SMOTE was applied only to the training data to compare oversampling against class weighting while keeping validation and test distributions unchanged.

The evaluation framework reports precision, recall, F1-score, ROC-AUC, PR-AUC, false positive rate, Brier score, confusion matrices, ROC and precision-recall curves, calibration curves, threshold

sweeps, feature importance, KS drift statistics, and cross-dataset performance degradation. RandomizedSearchCV was attempted with PR-AUC as the objective, but the cross-validation scoring output returned NaN values. Because of that issue, the report treats the search-selected estimators as final evaluated models and avoids claiming that tuning clearly improved performance.

4. Results

4.1 Exploratory data analysis

The final 2018 sample contained 76.4% benign traffic and 23.6% attack traffic, corresponding to an imbalance ratio of about 3.2:1. This is not an extreme imbalance, but attacks are still the minority class, so recall and PR-AUC are more meaningful than accuracy. The correlation heatmap showed that timing features were especially related to one another. For example, Flow IAT Min and Flow IAT Mean were nearly redundant, and Active Mean and Active Max moved together strongly. This became important later because the models also relied heavily on timing/IAT features, and those same features shifted between 2018 and 2017.

4.2 Same-year model performance on CSE-CIC-IDS2018

All three boosting models performed strongly on the 2018 test set. LightGBM had the best same-year PR-AUC among the base class-weighted models at 0.9178 and the best recall at 0.8515. XGBoost was extremely close, with PR-AUC of 0.9169 and recall of 0.8498. CatBoost was competitive but slightly lower on PR-AUC. The LightGBM + SMOTE variant did not clearly outperform class weighting, which suggests that class weighting was the more practical imbalance-handling method for this final setup.

Model / variant	Precision	Recall	F1	ROC-AUC	PR-AUC	FPR	Brier
LightGBM	0.8669	0.8515	0.8591	0.9546	0.9178	0.0404	0.0588
XGBoost	0.8707	0.8498	0.8601	0.9534	0.9169	0.0390	0.0587
CatBoost	0.8760	0.8453	0.8604	0.9521	0.9101	0.0370	0.0608
LightGBM + SMOTE	0.8669	0.8515	0.8591	0.9525	0.9155	0.0404	0.0585
LightGBM + Platt	0.9212	0.8334	0.8751	0.9530	0.9143	0.0221	0.0501
LightGBM + Isotonic	0.9393	0.8099	0.8698	0.9529	0.9078	0.0162	0.0489
XGBoost + Platt	0.9046	0.8393	0.8707	0.9522	0.9162	0.0274	0.0506

Table 2. Final same-year test performance. Lower FPR and lower Brier score are better; higher values are better for the other metrics.

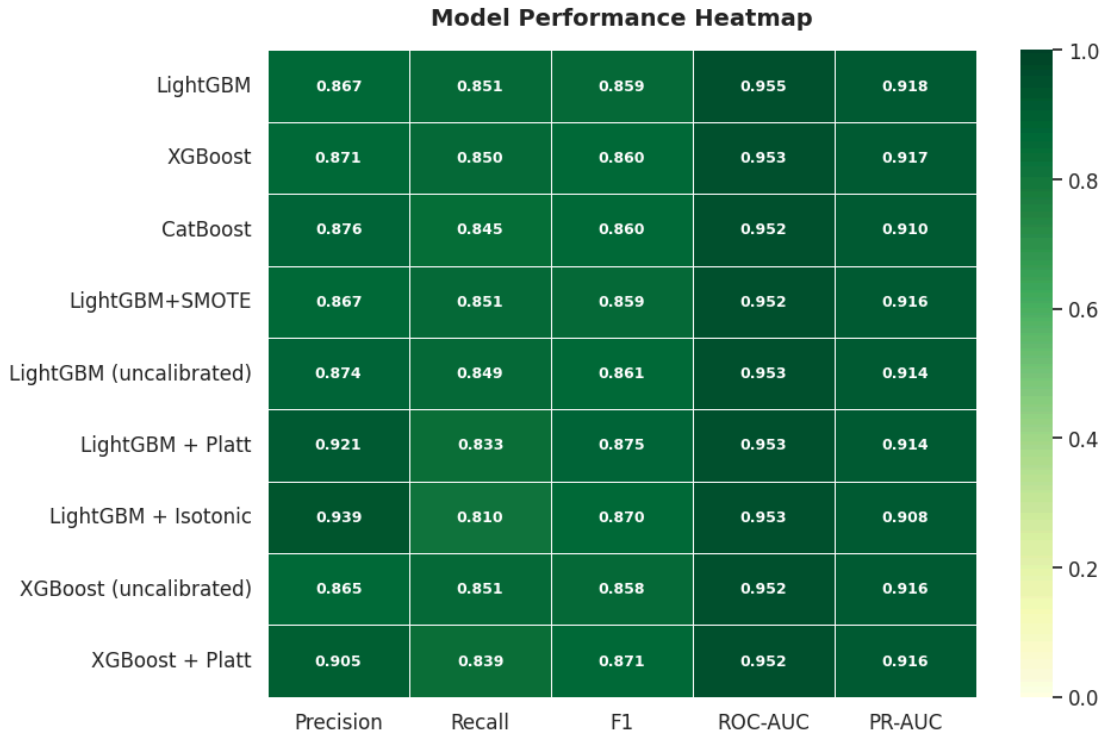

Figure 3. Model performance heatmap for final same-year test metrics.

Training time also supported LightGBM as the best speed/performance option. In the final notebook run, LightGBM trained in about 2.1 seconds, XGBoost in about 5.6 seconds, CatBoost in about 7.2 seconds, and LightGBM + SMOTE in about 5.7 seconds. This matters because intrusion detection systems often need repeated retraining and fast experimentation when traffic changes.

4.3 Confusion matrices and operational meaning

The default 0.5 threshold translated the model metrics into missed attacks and false alarms. On the 2018 test set, LightGBM missed 534 attacks, XGBoost missed 528 attacks, and CatBoost missed 547 attacks. XGBoost caught slightly more attacks at this threshold, while CatBoost created slightly fewer false positives. This shows why confusion matrices are valuable: they connect abstract metrics to actual security consequences.

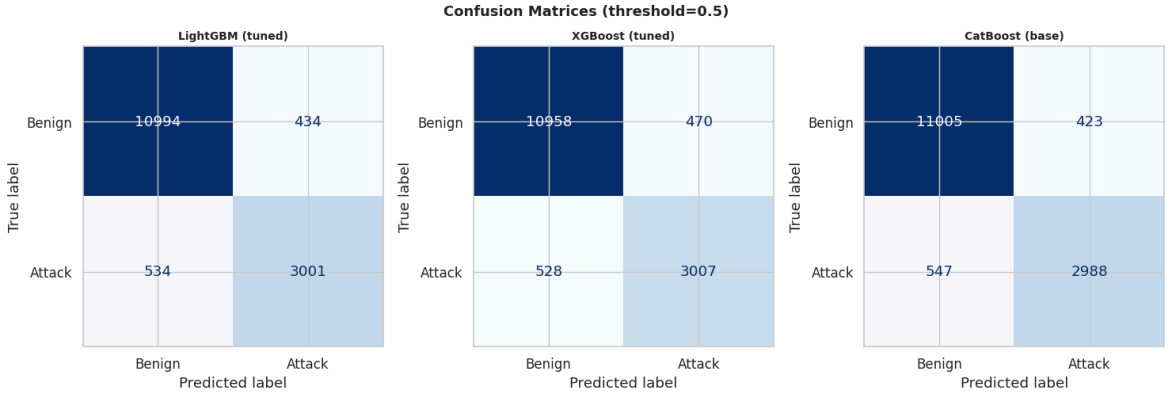

Figure 4. Confusion matrices for LightGBM, XGBoost, and CatBoost at threshold = 0.5.

4.4 Probability calibration

Probability calibration improved the reliability of risk scores. This matters because a probability output is only useful if it roughly matches the true chance of an attack. Brier score was used as the main probability-error metric, where lower values are better. LightGBM + Isotonic Regression produced the best Brier score at 0.04893, followed by LightGBM + Platt Scaling at 0.05010 and XGBoost + Platt Scaling at 0.05065. The uncalibrated LightGBM and XGBoost models had higher Brier scores, which suggests more probability error.

Calibration also changed threshold behavior. At the same default 0.5 threshold, calibrated models became more conservative: precision improved and false positive rate dropped, but recall decreased. For example, LightGBM + Isotonic reached precision of 0.9393 and FPR of 0.0162, but recall dropped to 0.8099. This tradeoff is acceptable only if the operational goal values fewer false alerts more than maximum recall.

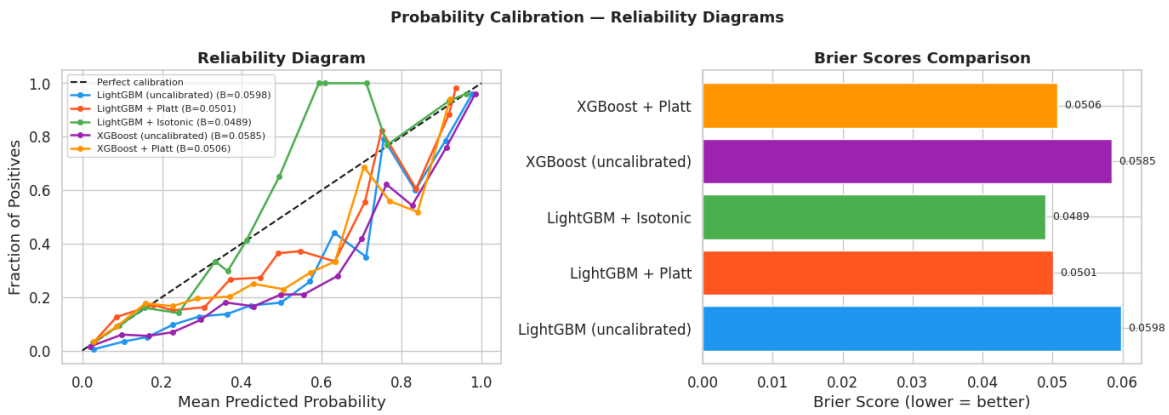

Figure 5. Calibration reliability diagrams and Brier score comparison.

4.5 Threshold optimization

The default 0.5 threshold was not treated as automatically optimal. A threshold sweep was run using the LightGBM + Platt calibrated model to show how precision, recall, F1, and false positive rate change as the alert threshold changes. The max-F1 operating point occurred at a threshold of about 0.709, with F1 = 0.8755, precision = 0.9334, and recall = 0.8243. This is a balanced setting for a security team with limited analyst capacity.

A high-recall operating point occurred at a threshold of about 0.035, with recall = 0.9675. However, the false positive rate rose to 0.4745, which would likely create too many alerts in a real security operations center. This result is important because it shows that catching more attacks is possible, but not free. The best threshold depends on risk tolerance and analyst workload.

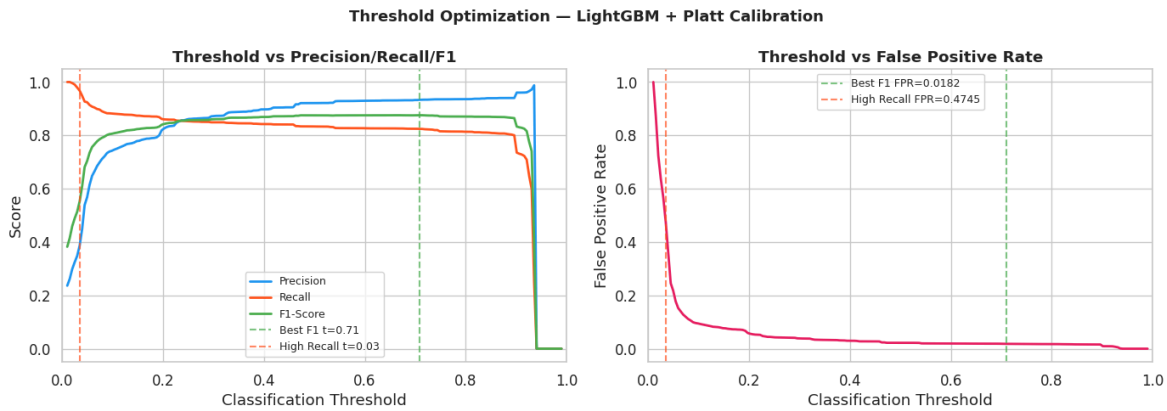

Figure 6. Threshold sweep showing the recall versus false-alarm tradeoff.

4.6 Drift analysis and cross-year robustness

The drift analysis exposed the biggest weakness in the final project. Models that performed well on the 2018 test split did not generalize cleanly to the 2017 dataset. The KS test showed that several timing/IAT features shifted strongly between years. Fwd IAT Min had the highest KS statistic at 0.3050, followed by Flow IAT Min at 0.2770, Fwd IAT Max at 0.2530, Fwd IAT Mean at 0.2517, and Flow IAT Mean at 0.1998. These were also among the features that appeared important for LightGBM, which helps explain why performance degraded under drift.

Model	PR-AUC 2018	PR-AUC 2017	PR-AUC drop	Recall 2018	Recall 2017	Recall drop
LightGBM	0.9143	0.2599	0.6543	0.8489	0.2005	0.6485
XGBoost	0.9162	0.3345	0.5817	0.8506	0.2248	0.6259
CatBoost	0.9101	0.2554	0.6547	0.8453	0.2132	0.6321

Table 3. Cross-year drift robustness summary. Lower drop is better.

XGBoost was the most robust model under drift because it had the smallest PR-AUC drop, falling from 0.9162 on 2018 test data to 0.3345 on 2017 data. However, all models degraded severely. LightGBM dropped from 0.9143 to 0.2599 PR-AUC, and CatBoost dropped from 0.9101 to 0.2554. This means strong same-year performance does not guarantee cross-year deployment robustness.

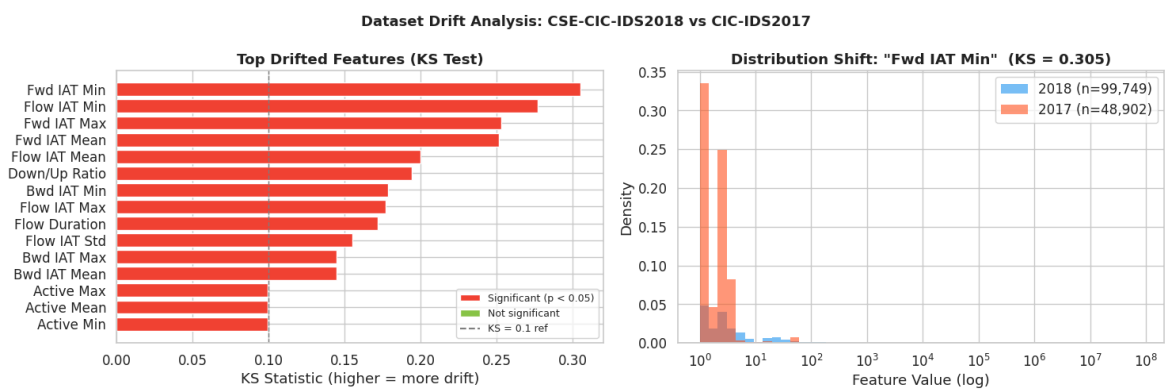

Figure 7. KS drift analysis between CSE-CIC-IDS2018 and CIC-IDS2017.

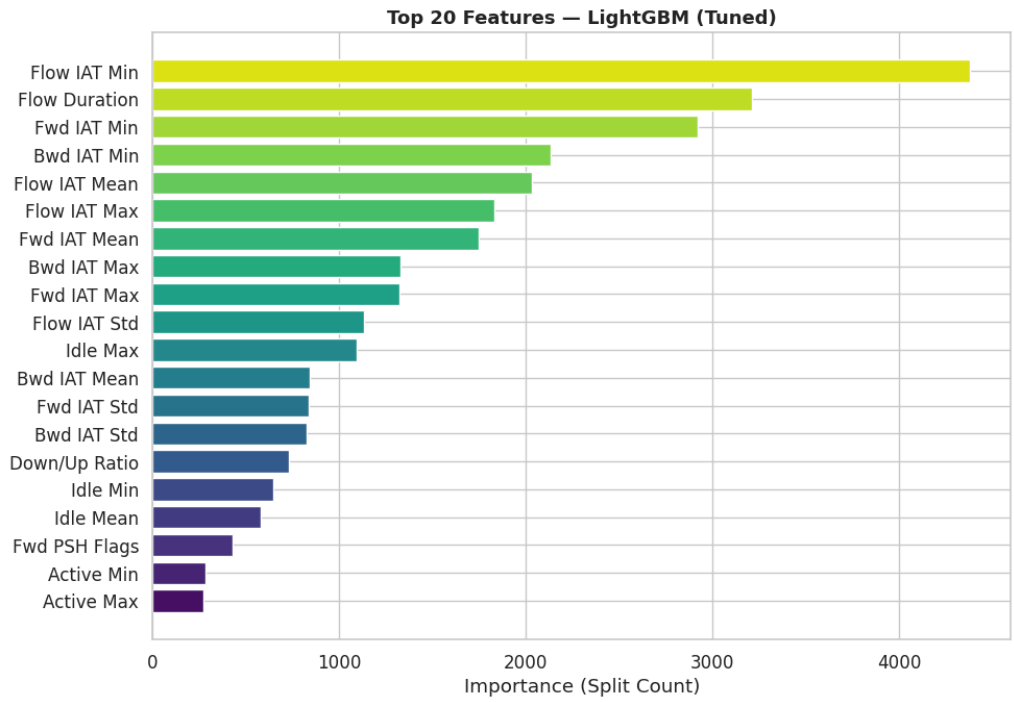

Figure 8. Top LightGBM features. Timing/IAT features were heavily used by the model.

5. Challenges and Limitations

Several limitations affected the final version of the project. The first was the size of the CSE-CIC-IDS2018 dataset. The full dataset was too large to repeatedly load and test in Google Colab without memory issues, so the notebook used chunked loading and a sampled subset of the data. This made the work manageable and reproducible, but it also means the results reflect a representative sample rather than the full dataset.

Another challenge was making the 2018 and 2017 datasets comparable. Although both datasets come from the CIC intrusion detection family, their columns and formatting do not match perfectly. To make the drift analysis fair, the final pipeline kept only the shared usable numeric features. This helped avoid feature mismatch problems, but it also reduced the amount of information available to the models. The final model also used binary labels, where all attack types were grouped into one general attack class. This made the project easier to evaluate and helped keep the comparison focused, but it also removed important differences between attack categories such as DDoS, bot, infiltration, and web attacks. In a real security setting, knowing the type of attack would be more useful than only knowing whether a flow is benign or malicious.

The train, validation, and test split was stratified and designed to avoid leakage, but it was still a random split within the 2018 dataset. This is useful for a controlled model comparison, but it is less realistic than a time-based split. In practice, a model would be trained on past traffic and then used on future traffic, where patterns may have changed. The project also showed that threshold selection is not simple. Lowering the threshold can catch more attacks, but it also creates many more false positives. The project demonstrated this tradeoff, but it did not select a final threshold based on a real analyst workload or daily alert limit. One issue in the final notebook was the hyperparameter tuning output.

RandomizedSearchCV was attempted using PR-AUC as the scoring objective, but the cross-validation PR-AUC returned NaN in the final run. Because of that, the results cannot not be presented as proof that tuning clearly improved the models.

The largest limitation was drift robustness. All models performed well on the 2018 test data, but performance dropped sharply when the same models were evaluated on the 2017 drift dataset. This suggests that strong same-year results do not guarantee reliable performance across time or across related datasets. For operational use, the model would need ongoing monitoring, recalibration, and retraining.

6. Future Work

Future work should focus on making the project more realistic for deployment and improving how well the models handle changing network traffic. The first step would be to fix the hyperparameter tuning issue and rerun the search with a valid PR-AUC scoring setup. That would allow a cleaner comparison between untuned and tuned versions of LightGBM, XGBoost, and CatBoost.

A stronger validation design would also improve the project. Instead of only using a random split, future work could train on earlier days of CSE-CIC-IDS2018 and test on later days. This would better represent how a model behaves when it is trained on past traffic and applied to future traffic. It would also help reveal drift problems before moving to the separate CIC-IDS2017 dataset. The project could also be extended from binary classification to multi-class attack classification. Instead of predicting only benign or attack, the model could try to identify specific attack categories. This would make the results more useful for incident response because analysts would receive more information about the likely type of threat.

Another improvement would be to run the pipeline on a larger sample or the full CSE-CIC-IDS2018 dataset using stronger computing resources. The current sample was necessary for Colab, but a larger run would reduce uncertainty and make the results more representative of the full benchmark. Future versions should also look more closely at feature drift. The results showed that timing and IAT-related features were important to the models and also shifted strongly between datasets. Testing drift-resistant feature engineering, alternative normalization methods, or training strategies that account for changing traffic patterns could help reduce the cross-year performance drop. For a more realistic security workflow, the threshold should be selected based on analyst capacity rather than only F1-score or recall. For example, the team could choose a threshold that limits alerts to a manageable number per day while still maintaining acceptable attack recall.

Finally, the pipeline could be packaged as a reusable workflow. This would include saving the preprocessing steps, trained model, calibration layer, selected threshold, and drift checks together. Doing this would make it easier for future teams to rerun the project, update the data, compare new models, and monitor when performance starts to degrade.

7. Conclusion

The final project successfully built a reproducible evaluation framework for gradient boosting models in network intrusion detection. LightGBM was the strongest same-year speed/performance

choice, with the best base PR-AUC and recall on CSE-CIC-IDS2018 while also training the fastest. XGBoost was nearly tied on same-year performance and was the most robust under cross-year drift, although its drifted performance still dropped sharply. CatBoost was competitive but did not lead the final comparison.

The most important result is that model choice alone is not enough. Calibration improved probability reliability, threshold tuning clarified the recall versus false-alarm tradeoff, and drift testing showed that strong same-year results can fail under shifted traffic. A practical intrusion detection system would therefore need model evaluation, probability calibration, threshold selection, and drift monitoring together before deployment.

8. Notes on AI Assistance

An AI assistant, such as ChatGPT, was used to support the writing and revision process for this report. Its role was limited to improving grammar, organization, clarity, and sentence flow so that our project ideas could be communicated more effectively. The underlying project ideas, methodology, implementation decisions, debugging process, testing, results interpretation, and analysis were completed by the team based on our own understanding of the project and course material. All core technical work, including dataset preparation, model implementation, evaluation, calibration, threshold analysis, and drift analysis, was conducted independently by the team.

9. References

Ahmad, Z., Khan, A. S., Shiang, C.-W., Abdullah, J., & Ahmad, F. (2021). Network intrusion detection system: A systematic study of machine learning and deep learning approaches. Transactions on Emerging Telecommunications Technologies, 32(1), e4150. https://doi.org/10.1002/ett.4150

Bouzaachane, K., El Guarmah, E. M., Alnajim, A. M., & Khan, S. (2025). Addressing modern cybersecurity challenges: A hybrid machine learning and deep learning approach for network intrusion detection. Computers, Materials & Continua, 84(2), 2391-2410. https://doi.org/10.32604/cmc.2025.065031

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785-794). ACM. https://doi.org/10.1145/2939672.2939785

Gharib, A. H., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP) (pp. 108-116). https://doi.org/10.5220/0006639801080116

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems, 30, 3146-3154.

Leevy, J. L., & Khoshgoftaar, T. M. (2020). A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 big data. Journal of Big Data. https://doi.org/10.1186/s40537-020-00382-x

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. Advances in Neural Information Processing Systems, 31.